Statistical Programming Languages Winter Semester 2025/2026
Take-Home Exam

KATARZYNA RELUGA

Due: 09 January 2026, 23:59

General Instructions

The take-home exam consists of the following five programming tasks.

1. **File Management:** Solve each exercise in a separate R-file, using `SPL25_surname_name_X.R` as a template. Rename the file accordingly (X corresponds to the number of the respective exercise) and fill in your names and student ID (Matrikelnummer) at the designated place in each file.
2. **Code Requirements:** Your R code:
   - must solve the exercises without errors or warnings when executed from top to bottom (exceptions are intended warnings or errors and ill-set working directories);
   - must provide self-contained solutions, i.e., the solutions to all exercises must be reproducible, obtained with R code, and each file must be executable on its own;
   - must include only one solution per task (otherwise only the first one will be considered);
   - must be comprehensible and include explanatory comments (also to structure the code and to answer specific questions);
   - must be your own intellectual work;
   - must not use any additional packages unless explicitly required/allowed in a specific exercise.
3. **External Sources/AI:**
   - If parts of the code are taken from external sources or AI tools, they have to be cited properly using comments.
   - If you use AI tools, you are required to provide all prompts used as well as their output in an appendix.
4. **Grading Penalties:**
   - Errors or warnings lead to 0 points for the respective task.
   - Using additional packages leads to 0 points for the respective task.
   - Any attempt to deceive results in the grade 'failed' (5.0).
   - Following this style guide is part of the grading.
5. **Submission:** You have to submit your solutions as **one ZIP file** by 09 January 2026, 23:59 via Moodle.

Please contact me via email (katarzyna.reluga@hu-berlin.de) if anything is unclear.

Grading Overview

Exercise	**1**	**2**	**3**	**4**	**5**	**Σ**
Points	3	17	25	25	30	100

GOOD LUCK!

You may use all packages loaded by default, namely: `stats`, `graphics`, `grDevices`, `utils`, `datasets`, `methods`, and `base`.

Exercise 1: Session Info [3 points]

Start a new R session and execute the following code:

```
sink("my_session.txt")
sessionInfo()
sink()
```

- Include the resulting file `"my_session.txt"` in your submission, i.e., in the zip file containing your solutions.
- Use comments to briefly explain the output provided by the function `sessionInfo()` in your own words.

Exercise 2: COVID-19 [17 points]

The file `corona.csv` contains information on the development of the COVID-19 pandemic in EU/EEA countries. Amongst others, the following variables were observed in the data set:

Variable (original name)	New name	Description
`dateRep`	`date`	Observation date
`cases`		Number of newly infected people on this date
`deaths`		Number of people who died from COVID-19 on this date
`countriesAndTerritories`	`country`	Country/Territory of the observation
`popData2020`	`population`	Population of the Country/Territory (as of 2020)
`indicator14`		Number of cases per 100,000 residents over the last 14 days (including current date)

a) Set your working directory appropriately and read the data conveniently into a data frame `corona`. Delete all variables not contained in the table above and rename the remaining ones according to *New name*, if *New name* is not empty for this variable. Transform the variable `date` to class `POSIXct` appropriately. Are the classes of the other variables represented adequately? Briefly justify your answer. **[2]**

b) How many observations in the data set contain missing values? **[1]**

c) How many observations in the data set counted less than 20 deaths? What's their share on the total number of observations? **[1]**

d) Use an appropriate test to decide whether the average value of `indicator14` differs in the countries Czechia and Slovakia in January 2022 (significance level: 5%). Briefly justify your decision on whether the average value differs. **[2]**

e) Compute the total number of deaths per country in September 2020 and sort them in descending order. **[3]**

f) In Germany, a 7 day indicator (`indicator7`) is used to evaluate the rate of new infections instead of `indicator14`. It is the number of cases per 100,000 residents over the last 7 days (including the current date) in one country, i.e.,

$$indicator7_i = \frac{\sum_{k=i-6}^{i} cases_k}{population_i} \cdot 100$$

for the i-th observation of the respective country. Create a new numeric variable `indicator7` containing this 7 day indicator and add it to the data frame `corona`.

Adhere to the following:

- Make sure that the order of observations in the data frame `corona` stays unchanged.
- For the first 6 observed days of a country, the new variable `indicator7` has the value `NA`.
- Always use the observations of the last 7 days existent in the data frame for your computation. (Some date-country-combinations are non-existent in the data frame, e.g., 9th January 2022 for Denmark.) **[6]**

g) Add a new factor variable `alert_level` to the data frame `corona` which is:

- "green" if `indicator7` is less than 35,
- "yellow" if `indicator7` is at least 35 but less than 50,
- "red" if `indicator7` is at least 50 but less than 100,
- "darkred" if `indicator7` is at least 100.

If you did not manage to solve exercise f) use the variable `indicator14` instead, but double the threshold values (e.g., use 70 as upper bound for "green"). **[2]**

Source: https://www.ecdc.europa.eu/en/publications-data/data-daily-new-cases-covid-19-eueea-country as of 12 April 2022.

Exercise 3: Airquality Graphic [25 points]

Construct the following graphic and save it as a PDF file in your working directory (using R code!), with the width and height of the graphic region set to 7 inches, respectively.

- The graphic includes everything inside the thick black frame (of the example image).
- It is a vector graphic, so you can zoom in on it.
- It is based on the built-in data set `airquality`. Use the variables `Solar.R` (Solar Radiation), `Ozone` (Mean Ozone), and `Month`.

Pay attention to the details, so your graphic looks as similar as possible. In particular:

- Arrangement of the plots and their proportion to each other (hint: `?layout`).
- Top panel: Histogram of Solar Radiation.
- Bottom Left panel: Scatter plot of Mean Ozone vs. Solar Radiation with different point symbols/colors for months.
- Bottom Right panel: Boxplot of Mean Ozone by Month.
- Width of cells in the histogram.
- Axis labels, scaling, and orientation/position of inscription (hint: `?axis`).
- Colors and point symbols.

Exercise 4: Data Cleaning [25 points]

A lot of common first names in Bavaria have specific nicknames. This can be confusing to people outside and inside of Bavaria. Read in the file `bavarian_nicknames_fem.csv` (UTF-8 encoded), which contains a non-exhaustive list of Bavarian names and their short forms. Unfortunately, the file needs some data cleaning.

Save the information as a `data.frame` named `nick`. Make sure that `nick` satisfies the following requirements:

- No empty lines, leading or tailing white spaces.
- The two character variables are named `Nickname` and `Name`, respectively (e.g. Agdi is a nickname of Agnes).
- All square brackets [] and their containing text are removed from the values in the variable `Name`.
- All shortening notation in the values of `Nickname` are replaced by a full notation of the alternatives (e.g. "Sieg-e/i, Sig(g)i, Sopherl/-Sofferl" is replaced by "Siege, Siegi, Sigi, Siggi, Sopherl, Sofferl").
- Each line contains only one name in the variable `Nickname` and there are no duplicated names in `Nickname`. The names of duplicated nicknames are merged.

Example of Merging:

Before:

Nickname	Name
Mirl, Mall	Maria
Mirl	Maria, Rosemarie

After:

Nickname	Name
Mirl	Maria, Rosemarie
Mall	Maria

The merging should not create duplicated names in the variable `Name` (i.e. Maria, Rosemarie instead of Maria, Maria, Rosemarie).
All of your data preprocessing steps have to be performed in R Base and pay attention to a good documentation of your steps. Write your code as general as possible, i.e. use regular expressions and refrain from explicit code such as manual editing. Your code will be applied to a separate data set that shows the same features and it will be graded based on the results on the separate data set.

Exercise 5: Battleship [30 points]

The game Battleship is played by two persons. To prepare the game, every player generates a grid (usually 10×10), which illustrates the player's combat area. Then, every player marks the position of their fleet of ships in their combat area, obeying the following rules:

- The ships may not touch horizontally or vertically (but they may touch diagonally!)
- Every ship has to be line-shaped (horizontally or vertically; not diagonally!)
- The ships may be located at the border (first/last row/column)

Usually, the fleet of ships consists of 10 ships:

- One battleship (5 cells)
- Two cruisers (4 cells each)
- Three destroyers (3 cells each)
- Four submarines (2 cells each)

Write a function `combat_area`, which randomly generates a combat area for a specified grid dimension and number of ships, respecting the rules above. It should fulfill the following:

1. The arguments of the function correspond to the number of rows (`n_row`; default: 10), to the number of columns (`n_col`; default: `n_row`), and to the number of ships of the respective length: `n5` (battleships, default: 1), `n4` (cruisers, default: 2), `n3` (destroyers, default: 3), and `n2` (submarines, default: 4).

```r
combat_area <- function(n_row = 10, n_col = n_row, n5=1, n4=2, n3=3, n2=4)
    {
    # Your part
}
```

2. The starting point is an empty combat area of the specified dimension ($n\_row \times n\_col$).
3. The ships are placed consecutively according to their size (starting with the largest).
4. To place a ship, start by determining its orientation (horizontal or vertical) randomly. Afterwards, choose a position at which the ship can be placed in the selected orientation (satisfying the rules above) randomly.
5. It is possible that there is no position left to place the next ship in the selected orientation. In this case draw a valid position for the remaining orientation.
6. If not even this is possible, the function should stop with the following error (appropriately adapted to the length of the respective ship; hint: `?stop` or `?stopifnot`):

```
Error in combat_area(): Ship of length 2 does not have enough space.
```

7. If the ships were placed successfully, the function returns a plot of the combat area (in the Plots-tab of RStudio).

Expected Output Appearance:

```
> combat_area()
Combat Area
  1 2 3 4 5 6 7 8 9 10
A           # # #
B
C #         # # # #
D #
E           # #
F   # #
G
H
I   # # # # #
J

> combat_area(n_row = 12, n_col = 13, n5=3, ...)
... (Grid adjusts to 12 rows A-L and 13 columns)
```

Hints:

- Outsource (self-contained) subproblems in separate functions, which are then called in the main function `combat_area`.
- Test the function with different calls. In particular, make sure that it also works for non-default parameters.